EfficientAD in Practice: An Introduction to Industrial Visual Anomaly Detection

A practical tutorial using the bottle category of MVTec AD, Anomalib, and Google Colab

Research paper: Kilian Batzner, Lars Heckler, and Rebecca König, “EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies,” WACV 2024.

Paper: https://arxiv.org/abs/2303.14535

Abstract

Industrial visual inspection must identify damaged, contaminated, or otherwise unusual products, including defects that may be rare or difficult to label in advance. Visual anomaly detection addresses this by learning the appearance of normal products and identifying deviations at inference time.

This tutorial introduces EfficientAD, a method designed to combine anomaly-detection accuracy with low inference latency. It explains the paper’s motivation and high-level architecture, then documents a preliminary experiment using the bottle category of the MVTec Anomaly Detection (MVTec AD) dataset. The experiment used Anomalib in Google Colab on an NVIDIA Tesla T4 GPU. EfficientAD-S was trained for 10 epochs using 209 normal training images and evaluated on a subset of 83 test images.

The recorded results were 97.59% image-level accuracy, 99.92% image-level AUROC, 97.23% pixel-level AUROC, and 90.15% AU-PRO at a false-positive-rate limit of 0.30. These are preliminary results on a partial subset, not official benchmark results. The tutorial concludes with limitations and a proposed path toward packaging inspection and edge deployment.

Keywords: EfficientAD, visual anomaly detection, industrial inspection, MVTec AD, Anomalib, PyTorch, anomaly maps, edge AI.

1. Introduction

Traditional supervised classification usually learns to distinguish a set of labelled classes. In industrial inspection, however, collecting examples of every possible defect can be expensive or impractical. A factory may encounter defects that are rare or previously unseen.

Visual anomaly detection offers another approach: learn what normal products look like, then flag images or image regions that differ from those learned patterns. For example, a model trained on normal bottles may flag a broken rim or unexpected contamination at test time.

Two outputs are useful:

- **Image-level detection** estimates whether an image as a whole appears anomalous.
- **Pixel-level localization** produces an anomaly map showing where unusual regions may be located.

An anomaly score is not automatically a probability. It is a numerical measure of how unusual an input appears. A decision threshold is used to turn scores into normal/anomalous predictions.

2. The EfficientAD paper at a glance

Batzner, Heckler, and König introduced EfficientAD to address the need for accurate industrial anomaly detection at low latency. The method combines an efficient feature extractor with a student–teacher approach and an autoencoder-based branch that analyses broader image context. The authors evaluate EfficientAD across industrial anomaly-detection datasets and report millisecond-level inference on modern GPU hardware.

- Paper: https://arxiv.org/abs/2303.14535
- Published version: WACV 2024, pp. 128–138: https://openaccess.thecvf.com/content/WACV2024/html/Batzner_EfficientAD_Accurate_Visual_Anomaly_Detection_at_Millisecond-Level_Latencies_WACV_2024_paper.html

2.1 Core idea and architecture

EfficientAD learns from defect-free training images. At inference time, it compares feature representations produced by different parts of the model. Disagreement can indicate an anomaly.

The main components are:

- **Teacher network:** provides target visual features. In the paper, the teacher uses a compact Patch Description Network (PDN) distilled from a larger pretrained network.
- **Student network:** learns to predict the teacher's features on normal training images. At test time, it may fail to match the teacher around anomalous regions.
- **Autoencoder branch:** learns normal feature patterns and helps identify certain global or logical anomalies—unusual combinations or arrangements that may not be apparent from a small local patch alone.

The student–teacher comparison helps identify local texture or structural deviations. The autoencoder branch adds broader context. The anomaly signals are calibrated and combined into a spatial anomaly map and an image-level score.

2.2 Why efficiency matters

Some anomaly-detection methods require expensive searches through stored feature vectors or use computationally heavy networks. EfficientAD is designed to reduce inference cost with lightweight feature extraction and a compact student–teacher model. This makes it a candidate for applications where inspection must happen quickly.

The latency reported in the paper was measured under the authors' experimental conditions. It should not be assumed to apply to every GPU, CPU, input resolution, or edge device. Deployment performance must be measured on the intended hardware.

3. Experiment overview

The goal was to validate a practical end-to-end workflow: prepare data, train EfficientAD-S, generate predictions, inspect anomaly maps, and calculate image- and pixel-level metrics.

This was an initial implementation using Anomalib's EfficientAD implementation. It was **not** a full reproduction of every setting in the research paper.

3.1 Environment

Component	Configuration
Environment	Google Colab
GPU	NVIDIA Tesla T4
Python	3.13
PyTorch	2.11.0+cu130
Torchvision	0.26.0+cu130
Anomalib	2.7.0
Model	EfficientAD-S
Requested training duration	10 epochs
Training batch size	1
Evaluation batch size	4
Data-loader workers	2
Random seed	42

Colab package versions can change. Record the versions actually available and check compatibility before reproducing the experiment.

3.2 Dataset

The experiment used the **bottle** category from a Hugging Face mirror of MVTec AD. The original dataset download route attempted through the library returned HTTP 404 during this run, so a mirror was used.

Split/category	Images
`train/good`	209
`test/good`	20
`test/broken_large`	20
`test/broken_small`	22
`test/contamination`	21
Total test images	83
Ground-truth masks for defective test images	63

Only normal images were used for training. Defective examples were reserved for testing. Ground-truth masks supported evaluation of defect localization.

This was a partial subset/mirror, not the complete official MVTec AD bottle benchmark. Results must be reported as subset results and should not be compared directly with published full-benchmark scores. Review the dataset's current licence and usage conditions before redistribution or commercial use.

4. Step-by-step implementation in Google Colab

The workflow below documents the experiment. Exact dataset URLs and package dependencies can change, so use the saved notebook as the source of truth for the original code cells and verify the directory structure before running them again.

Step 1 — Create a GPU-enabled Colab notebook

Open Google Colab, create a notebook, and select **Runtime → Change runtime type → GPU**. Then verify the runtime:

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

The experiment ran on an NVIDIA Tesla T4.

Step 2 — Install and verify dependencies

The recorded environment used Anomalib 2.7.0, PyTorch 2.11.0+cu130, and torchvision 0.26.0+cu130. Install compatible dependencies in a fresh runtime, then verify imports. Colab's preinstalled PyTorch/CUDA combination may differ, so avoid mixing incompatible binary builds.

```
import torch
import torchvision
import anomalib

print("PyTorch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("Anomalib:", anomalib.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Step 3 — Download and inspect the dataset

The experiment used `huggingface_hub.snapshot_download` to obtain the bottle category from a Hugging Face mirror. The resulting directory needed to include normal training images, test categories, and ground-truth masks in a structure compatible with Anomalib's MVTec AD data module.

Simplified expected structure:

```
bottle/
├── train/
│   └── good/
├── test/
│   ├── good/
│   ├── broken_large/
│   ├── broken_small/
│   └── contamination/
└── ground_truth/
    ├── broken_large/
    ├── broken_small/
    └── contamination/
```

Verify file counts and the actual directory layout after downloading; mirrors can change. The original dataset download route returned HTTP 404 during this experiment. Use the exact source recorded in the notebook and check its licence.

Step 4 — Configure the data module

Anomalib's MVTec AD data module handles image loading, transforms, batching, and the train/test split. Configure it for the **bottle** category and the downloaded dataset root. The experiment used training batch size 1, evaluation batch size 4, and two data-loader workers.

The exact constructor arguments depend on the installed Anomalib version and the path used in the notebook. Use the saved notebook's data-module cell rather than guessing version-specific arguments. Inspect a sample batch before training.

Step 5 — Initialise EfficientAD-S

The model was initialised through Anomalib:

```
from anomalib.models import EfficientAd
```

```
model = EfficientAd(
    model_size="small",
    imagenet_dir="/content/datasets/imagenette",
)
```

The `imagenet_dir` argument points to auxiliary data used by this implementation. During setup, the experiment downloaded pretrained teacher weights and Imagenette data, calculated teacher-feature statistics and validation quantiles, and then proceeded to training. These details are specific to the implementation and version used.

Step 6 — Train the model

Anomalib's `Engine` coordinates training, validation, checkpointing, and prediction. The experiment used one GPU, a maximum of 10 epochs, and `/content/results/efficientad_bottle` as the results root.

```
from anomalib.engine import Engine

engine = Engine(
    max_epochs=10,
    accelerator="gpu",
    devices=1,
    default_root_dir="/content/results/efficientad_bottle",
)

engine.fit(model=model, datamodule=datamodule)
```

Here, `datamodule` is the data module configured in Step 4. Use the matching variable names and arguments from the saved notebook. Training completed after the requested 10 epochs.

Step 7 — Locate and preserve the checkpoint

The checkpoint was saved at:

```
/content/results/efficientad_bottle/
  EfficientAd/MVTecAD/bottle/v1/
  weights/lightning/model.ckpt
```

It was approximately **73.4 MB**. Colab's local storage is temporary, so copy the checkpoint and important results to persistent storage before disconnecting or resetting the runtime.

Step 8 — Run inference

Predictions were generated with the trained checkpoint:

```
predictions = engine.predict(
    model=model,
    datamodule=datamodule,
    ckpt_path=checkpoint_path,
)
```

`checkpoint_path` points to the saved `model.ckpt`. The call returned 21 batches covering 83 test images at evaluation batch size 4.

Prediction batches included fields such as:

- `image_path`: source image location
- `pred_score`: image-level anomaly score
- `pred_label`: predicted normal/anomalous label
- `anomaly_map`: spatial anomaly scores
- `gt_label` and `gt_mask`: ground-truth labels and masks where available

The first batch's anomaly map had shape `4 × 256 × 256`. A high-scoring region indicates a potentially unusual region, not a confirmed defect.

Step 9 — Inspect anomaly maps

Anomaly maps show where the model believes unusual patterns occur. Compare each map with the original image and ground-truth mask.

In the first four `broken_large` examples, the model responded around parts of the damaged bottle rims, but the predicted regions did not perfectly match every ground-truth defect. An image can be classified correctly even when its localization is imperfect.

Inspect false positives, false negatives, small defects, and normal images under varied lighting and positioning—not just a few successful examples.

Step 10 — Calculate evaluation metrics

The experiment calculated image-level accuracy, balanced accuracy, image-level AUROC, pixel-level AUROC, and AU-PRO.

- **Accuracy:** proportion of images classified correctly at the chosen threshold.
- **Balanced accuracy:** average of sensitivity (fraction of defective images detected) and specificity (fraction of normal images correctly accepted).
- **AUROC:** measures how well scores rank anomalous examples above normal examples across thresholds. Image-level AUROC uses one score per image; pixel-level AUROC uses anomaly-map scores and pixel labels.
- **AU-PRO:** evaluates overlap between predicted anomalous regions and ground-truth defect regions while accounting for false-positive pixels. Here it used a false-positive-rate limit of 0.30.

Anomaly scores are not calibrated probabilities. Accuracy depends on the threshold selected by the model configuration; AUROC evaluates ranking across thresholds.

5. Results

5.1 Overall metrics

Metric	Result on 83-image subset
Image-level accuracy	97.59% (81/83)
Balanced accuracy	96.67%
Image-level AUROC	99.92%
Pixel-level AUROC	97.23%
AU-PRO at FPR limit 0.30	90.15%

5.2 Per-category results

Test category	Images	Correct	Accuracy	Mean anomaly score
`broken_large`	20	20	100.0%	0.5118
`broken_small`	22	22	100.0%	0.5143
`contamination`	21	20	95.2%	0.5822
`good`	20	19	95.0%	0.4983
Total	83	81	97.59%	—

Mean anomaly scores are descriptive values, not probabilities.

5.3 Confusion matrix

Rows are actual classes and columns are predicted classes. The class order is normal, anomaly.

Actual / predicted	Normal	Anomaly
Normal	19	1
Anomaly	1	62

There was one false positive among 20 normal images and one false negative among 63 defective images.

5.4 Interpretation

On this small subset, the model ranked normal and defective images well and produced useful localization signals. However, the anomaly maps did not perfectly outline every defect. This is why image-level detection and pixel-level localization must be evaluated separately.

These results do **not** establish production readiness. The test subset is small and may not represent factory conditions. More representative data, repeatable evaluation, threshold analysis, failure analysis, and testing under realistic lighting, camera, and product variation are needed before operational claims can be made.

6. Applying the approach to defective packaging

The bottle experiment demonstrates a workflow that could be adapted to packaging inspection, but it does not itself validate packaging performance.

A next phase could be:

- Define the packaging type and defects of interest, such as damaged cartons, torn pouches, visible seal defects, or label damage.
- Collect representative images using the intended camera angle, lighting, distance, and production conditions.
- Train on normal examples and reserve defective examples for validation and testing.
- Evaluate both image-level detection and defect localization; inspect false alarms and missed defects.
- Select decision thresholds based on the cost of missed defects versus unnecessary rejections.
- Test on the intended edge hardware and measure latency, throughput, memory use, and prediction consistency.

EfficientAD can flag visible deviations, but cannot guarantee detection of defects that are invisible in an image. Tasks such as exact text verification, hidden-leak detection, or internal seal validation may need additional inspection methods.

7. Feature Use Case : Automated visual detection inspection

Objective: Detect and localize visible defects in manufactured products using normal product images as the primary training data.

EfficientAD can be used as the anomaly-detection component of an automated visual inspection system. During training, the model learns the visual characteristics of defect-free products. During inference, it assigns an anomaly score to each input image and generates an anomaly map highlighting regions that differ from learned normal patterns. This is particularly useful when defect types are diverse or difficult to enumerate in advance.

For packaging inspection, a potential implementation could identify:

- Damaged or deformed cartons and containers.
- Visible contamination or unusual surface marks.
- Tears, irregularities, or damage around packaging edges.
- Abnormal label placement or other visible structural deviations, subject to validation.

The system could flag suspicious products for rejection or further human inspection. However, the ability to detect each defect type must be established through testing on representative packaging images.

8. Where the Feature can be Leveraged

The approach is relevant to several industrial applications:

Application	Potential use
Food and consumer-goods packaging	Detect visible damage, contamination, and surface irregularities.
Pharmaceutical packaging	Flag visible packaging damage and unusual blister-pack patterns.
Manufacturing quality control	Identify unexpected surface defects in components and finished products.
Production-line inspection	Inspect products continuously and flag suspicious items for further review.

9. Edge deployment: proposed next steps

The present experiment ran on a Tesla T4 in Colab; it did **not** test deployment on a Raspberry Pi or NVIDIA Jetson device.

A careful deployment plan is:

- Use the existing trained checkpoint to investigate export, rather than retraining immediately on packaging data.
- Check the installed Anomalib version's export options and test ONNX export if supported.
- Verify that exported inference produces sufficiently consistent anomaly scores and maps compared with the original model.
- On the actual target device, measure warm-up and steady-state latency, throughput, memory use, and CPU/GPU utilisation.
- If using compatible NVIDIA hardware, investigate TensorRT acceleration and verify output consistency after optimization.
- Repeat performance and detection tests at the intended image resolution and inspection rate.

Successful export on Colab does not prove that the model will run at the required speed on edge hardware. The target device and operational constraints must be known before making that claim.

10. Limitations

- The experiment uses a partial MVTec AD bottle subset, not the complete official benchmark.
- The test set contains only 83 images, limiting statistical confidence.
- Training was limited to 10 requested epochs; this was pipeline validation, not a comprehensive hyperparameter study.
- Only one run and a small set of visualizations were analysed.
- No edge device was benchmarked.
- Performance on bottles does not guarantee performance on other products, packaging materials, lighting conditions, or defect types.

These limitations should accompany the results whenever they are presented publicly.

11. Conclusion

This tutorial introduced EfficientAD and documented an initial experiment using EfficientAD-S through Anomalib in Google Colab. The model was trained on 209 normal bottle images and evaluated on 83 test images from a partial MVTec AD subset. The recorded results were 97.59% image-level accuracy, 99.92% image-level AUROC, 97.23% pixel-level AUROC, and 90.15% AU-PRO at an FPR limit of 0.30.

The experiment exercised the core workflow—data preparation, training, checkpointing, inference, anomaly-map inspection, and metric calculation. At the same time, the limited dataset and imperfect localization mean that results should be treated as preliminary.

The next stage is to study failure cases, validate on a more representative dataset, and test export and inference on the actual edge hardware. Only then can the approach be assessed for real-world packaging inspection.

References

- Batzner, K., Heckler, L., & König, R. (2024). *EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies*. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 128–138. https://arxiv.org/abs/2303.14535

- Open-access paper page: https://openaccess.thecvf.com/content/WACV2024/html/Batzner_EfficientAD_Accurate_Visual_Anomaly_Detection_at_Millisecond-Level_Latencies_WACV_2024_paper.html

- Anomalib implementation and documentation: https://github.com/open-edge-platform/anomalib